AI for Fair Transition - QUANTOS S.A. Internship

Kleon Karapas

July 2025 - October 2025

1 Acknowledgments

I would like to express my sincere gratitude to the entire AI4FT team, and especially to Vagelis Karapetsas, Eirini Mantzouni, and Christos Angelidis, all of whom are exceptionally skilled data analysts. Vagelis Karapetsas served as the team's project manager, and throughout the internship I reported to both him and to Photis Stavropoulos, whose guidance and oversight were invaluable.

To Vagelis, Eirini, and Christos, I am deeply grateful for their patience, their willingness to teach me, and for giving me the space and trust to contribute meaningfully to the project. Their openness allowed me to add new features, experiment with ideas, and implement improvements that strengthened the pipeline. I truly appreciate their support, collaboration, and the positive environment they created throughout my time on the project.

2 Company presentation and Project

Quantos SA Statistics and Information Systems is a Greek company established in 1997 with the vision to offer high-quality scientific consultancy services in statistics and data analytics in general, and supporting IT solutions. Quantos' business strategy has two axes: a) the provision of consulting services to public and private organizations worldwide (e.g., the European Commission, African Development Bank, OECD, Bank of Greece, Hellenic Statistical Authority); b) the development of data analytics services and products based on basic and industrial research. Its consulting services cover the complete lifecycle of quantitative information: from the conception of data products and sample surveys, through data collection, processing, privacy protection treatment, analysis and visualisation, to the recommendation of actions based on the results of analysis. They also include evaluation of corresponding methods and practices of third parties, and development of bespoke software tools for their implementation.

The AI for Fair Transition project (AI4FT) is a system developed for the European Commission (DG EMPL: Directorate-General for Employment, Social Affairs and Inclusion). Its purpose is to collect, analyze, and classify public policies related to fair, green, and digital transition across EU Member States. It starts by centralizing thousands of policy documents, then extracts structured information using AI and finally classifies each policy according to European Commission-defined categories (which will be detailed later). Finally, it produces statistics which are shown in interactive graphs whose values are changed according to the categories selected by the user.

3 System Architecture

During the first two weeks I focused on understanding the codebase and the different components and infrastructure involved (see Fig. 1). This taught me a lot of things regarding system design, and the choices software engineers make to fit also the required product's specifications. I will first start by exposing what are the main workers involved in the pipeline from collecting the policy documents to showing the interactive graphs in the frontend.

The first worker is named the **Crawler** and it is the entrypoint of the system focusing on document collection. It automatically scrapes documents from official government portals, ministries, and EU-related publication sites. It downloads the corresponding raw files and stores them in Azure Blob Storage. To ensure every file is tracked and no duplicates can enter the system once the crawling process runs again, a unique identifier

is generated and stored along with the timestamp and URL in Postgres.
Once the document is downloaded, the **Parser** worker processes it. It checks whether the file is readable (non-corrupted PDF, valid text content etc.), it matches minimum content length, structure constraints and contains the required policy information. Some documents are in national languages (DE, ES, FR, IT, PL, SE, etc.) so the parser detects these and sends them to the European Commission's internal translation API.
The **Extractor** worker then sends these documents to the European Commission's internal LLM API, where an AI model extracts structured information. This is done by categorizing the policies in the respective categories established by the European Commission:

- Document metadata: title, year, country, language, territorial scope
- Policy measures inside each document
- Types of intervention: Quality employment and job creation, Education, training and life-long learning, Fair-tax benefit systems, social protection systems, Transport, Energy, Access to essential services, Climate and green transition, Cross-cutting
- Policy packages
- Implementation status categories: Completed, In effect, Not applicable, Planned
- Target groups

Output is stored in document-level and measure-level fields in MongoDB.
This information is then retrieved from MongoDB and transformed into meaningful statistics and graphs by the **Frontend** worker (see Fig. 3). The cleaned data is then displayed in an interactive dashboard made using the Streamlit python library. The user can filter policies by country, aspect intervention type and policy package (see Fig. 2)

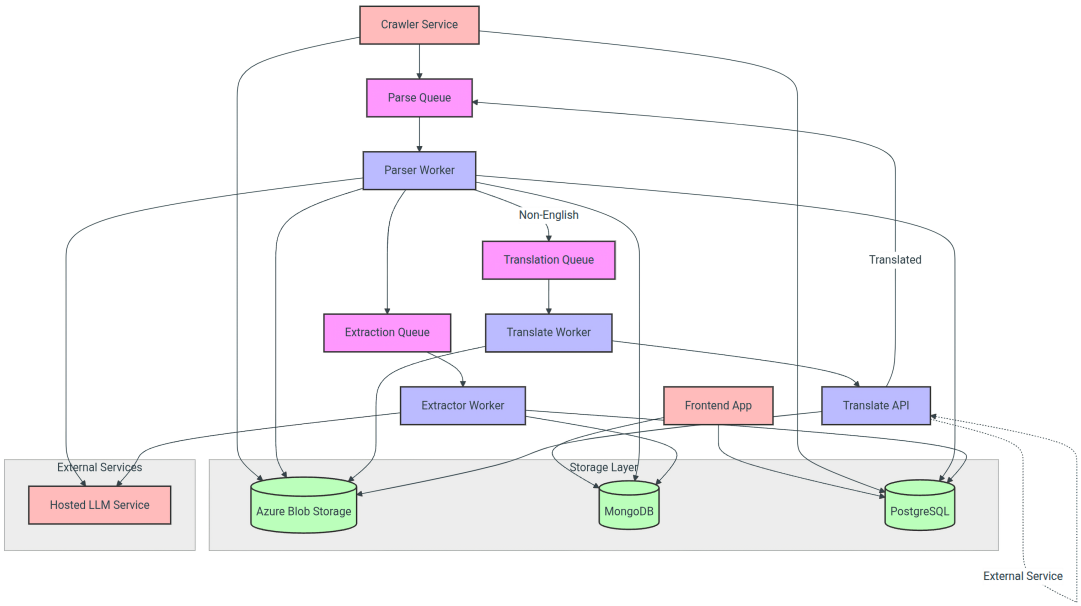

Figure 1: System architecture showing the full pipeline along with the queues and databases

4 Tasks and Acquired Skills

4.1 Deployment of the System Components and Infrastructure Setup

One of my first major responsibilities during the internship was the deployment of all system workers. The EU Commission mandated a strict infrastructure model: all services had to be deployed as Docker

containers running inside Commission-managed Azure virtual machines (VMs). Because the 4 workers were containerized, Docker became an essential skill. I had to learn how to write Dockerfiles optimized for production (minimizing image size, defining correct entrypoints, avoiding unnecessary layers), how to manage multi-container deployments with Docker Compose, how to structure code so that each container remains an independent microservice and finally how to handle permission packages, user groups, and file permissions inside containers to comply with Commission security policies. Working with Docker made clear why microservice-based architectures are powerful: each unit of the system is isolated, reproducible, and independently deployable. This modularity was critical because different parts of the pipeline used different dependencies (Python, Scrapyd, Streamlit, MongoDB clients, etc.), and Docker ensured consistency across development and production environments.
The European Commission imposed strict security rules. One of the consequences was that I did not have direct access to web interfaces exposed by containers, because the external ports of the VMs were closed, even when they were necessary for debugging and monitoring. For example, the ScrapydWeb UI was blocked, even though it was essential to inspect crawler jobs and diagnose failures. To solve this, I designed and implemented a solution using Azure Bastion, which provides secure SSH access without exposing public IP ports. The idea was to connect to a VM through Azure Bastion, then open an SSH tunnel between my local machine and the VM and forward the internal ports (e.g., 8501 for Streamlit frontend, 6800/6801 for ScrapydWeb) to localhost on my laptop. This allowed me as well as the whole team to safely access the ScrapydWeb job management interface and the user-facing Streamlit dashboard. This approach respected all Commission requirements because no ports were opened publicly, and all access was routed through an encrypted, authenticated Azure Bastion connection. Another non-technical but crucial responsibility was controlling cloud costs. Azure resources can become expensive if poorly configured, especially when running multiple VMs for different services. I analyzed the CPU and RAM usage of each microservice (crawler, parser, extractor, frontend) and determined the optimal VM size for each one.

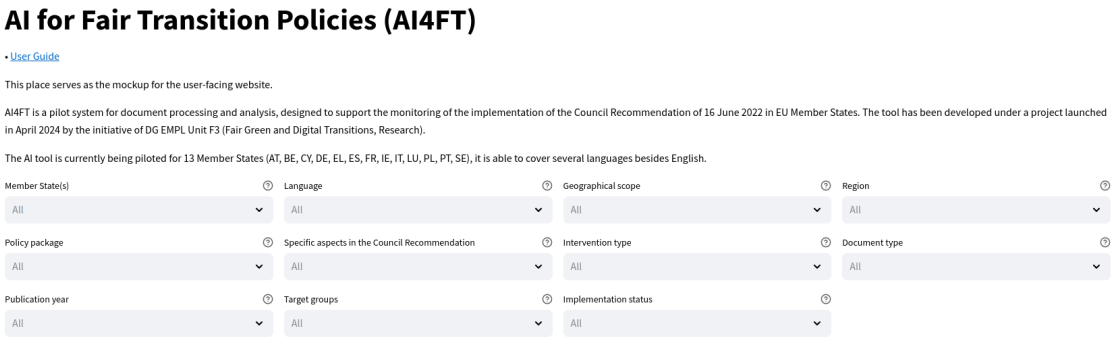

Figure 2: The figure above is taken from the Streamlit-based frontend dashboard developed for the AI4FT system. It displays the full set of policy filters available to users. These same dimensions are used internally by the Extractor worker during the classification process, meaning that the filters exposed to users directly correspond to the structured categories produced by the AI-based document analysis pipeline.

4.2 Automating the Crawling process using Cron

The AI4FT crawling worker is a distributed web scraping system built on Scrapy (for scraping) and Selenium (for browser automation), deployed as a containerized Scrapyd microservice managing over 90 specialized spiders in 13 European countries. Crawled documents flow through a processing pipeline that performs country and date filtering, SHA-512 deduplication to make sure that the same documents are not scraped twice, and metadata storage in PostgreSQL before uploading files to Azure Blob Storage and enqueuing parsing tasks via Azure Queues. The architecture supports both static HTML and JavaScript-rendered content through headless browser automation using Selenium, with comprehensive error tracking and a ScrapydWeb monitoring interface (see Fig. 4).
At the end of my Docker and Azure infrastructure work, the team asked me to design a solution that would allow all spiders to run automatically once per month without requiring manual access to the VMs

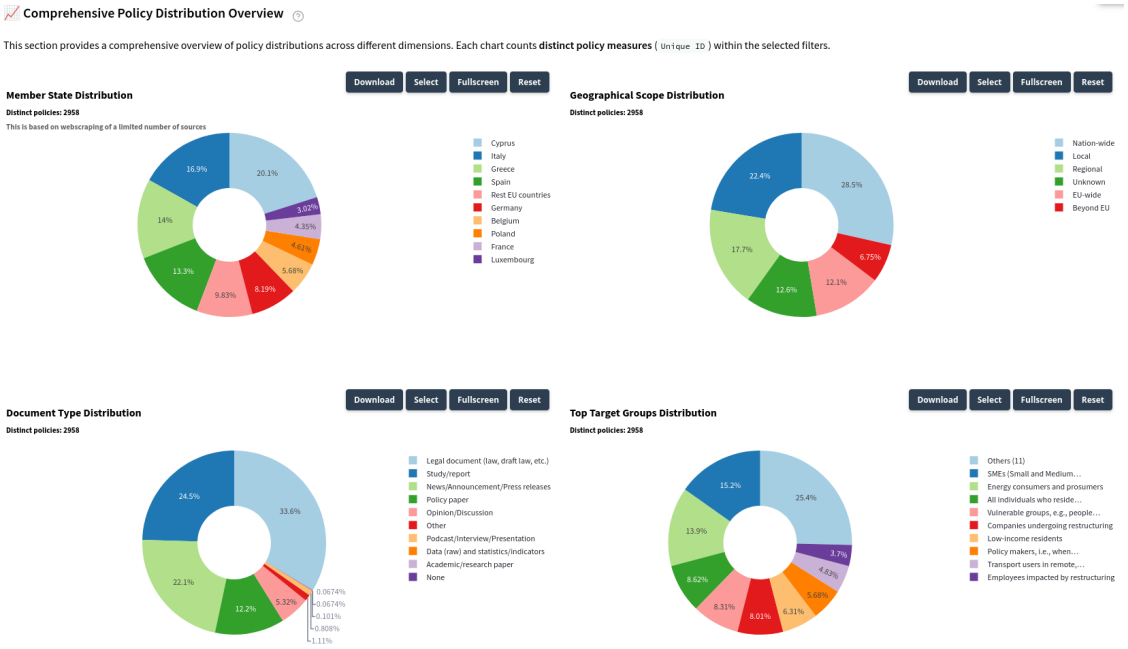

Figure 3: Interactive summary plots: the plot changes according to the selected filters from above in Fig. 2

or the Docker containers. My proposal was to implement a Cron-based scheduled execution within the VM, triggering Scrapyd job submissions on a monthly interval, ensuring hands-off autonomous operation. Working on this component taught me valuable skills, including managing distributed scraping systems, orchestrating automated tasks inside containerized environments, configuring Scrapyd and ScrapydWeb, and troubleshooting Selenium-based headless browser sessions. I also learned how to combine traditional Linux scheduling tools like Cron with modern microservice deployments to achieve secure, repeatable, and fully automated pipelines.

4.3 Improving Context Retention in Parser with Hierarchical Tree Structure

The Parser worker is responsible for converting raw policy documents into structured, semantically rich data suitable for downstream extraction and classification. It operates between the crawling layer and the LLM-based extraction layer, transforming unstructured PDFs or text files into hierarchical JSON trees enriched with multi-level embeddings. This transformation enables more accurate policy analysis, semantic search, and categorisation across EU Member States by the Extractor in the next stage of the pipeline.
The most impactful part of my work on the parser was designing a tree-like structure used to reassemble chunk-level results into a coherent document representation. Although the existing pipeline correctly chunked and processed text, it lacked a mechanism to reliably reconstruct the semantic relationships between sections once the document had been fragmented. Chunking is necessary because LLMs have context limits, but it introduces a major problem: once a document is split into fragments, the language model loses the global context. Policy documents depend heavily on their internal hierarchy (titles, subsections, paragraphs), so flattening them into isolated chunks degrades both structure and extraction quality. This algorithm works in the following way. For documents converted from PDF to Markdown, the system first detects Markdown headers (e.g., **#**, **##**, **###**) in order to identify the structural hierarchy of the original document. The extracted text is then grouped under each corresponding header, allowing the parser to maintain section-level organization. Oversized sections are recursively subdivided into smaller segments to comply with the chunk-size constraints of the pipeline.
By providing each chunk with its original header, the language model is able to infer the fragment's role within the broader document structure, even though it processes the text in isolation. This contextual information ensures that the model understands where the chunk belongs semantically, which is essential for reconstructing the overall hierarchy of the document. Once all chunks have been processed, the algorithm assembles the final document representation. During aggregation, chunks that share the same header are

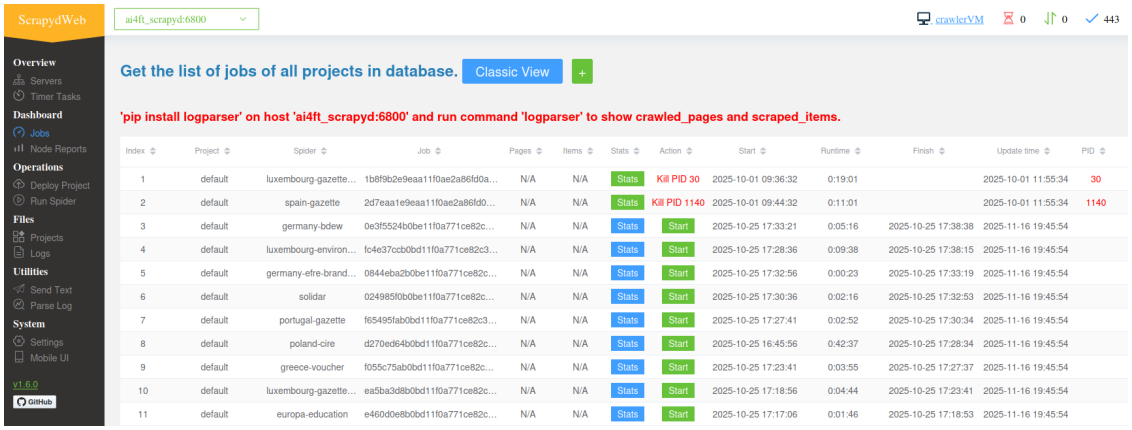

Figure 4: The figure above shows the ScrapydWeb user interface used to monitor and manage all spider jobs running inside the AI4FT crawling microservice. The dashboard provides a centralized view of all deployed spiders, including their status, runtime, start/finish timestamps, and log access. In the top-right corner, the interface displays global statistics such as the total number of completed spider jobs (443 in this case), real-time performance indicators, and access links to system logs. Each row in the jobs table represents the execution of a specific spider for a given Member State, with quick access to the job's logs, metadata, and controls to restart or terminate the job if necessary.

grouped together and identified as belonging to the same section. The child nodes extracted from these chunks are then merged, allowing the system to consolidate substructures that were originally split across multiple fragments. This merging process results in the construction of a unified document tree that accurately reflects the hierarchical organisation of the original text. Importantly, the system also preserves complete traceability: every node in the tree carries indices that link back to the specific chunks and therefore the precise character offsets from which it was derived. This part of the project was the most interesting to me because it connected directly to my university's Algorithms course which I really liked, where tree data structures were a recurring topic.

4.4 Training the Team in Modern Git Collaboration and Code Review

One of my final responsibilities during the internship involved supporting the development of the Streamlit-based frontend dashboard, particularly as the team planned to extend it with additional visualisations and analytical plots. Since multiple people would be contributing code simultaneously, I took the initiative to teach the team the full capabilities of Git, including the use of branches, structured merging practices, and the correct workflow for submitting and reviewing pull requests. I introduced a branching strategy adapted to our project, explained how feature branches should be created and isolated, how merge conflicts can be resolved safely, and what quality-control steps (linting, testing, peer review) should occur before approving a pull request into the main branch. This task was particularly enjoyable for me because it allowed me to apply and test my managerial and organisational skills. I had the opportunity to coordinate the team's workflow, clarify responsibilities, and help everyone collaborate more efficiently on a shared codebase. It also became a meaningful moment of team building: the developers were open-minded, eager to improve their practices, and trusted me to guide the process. I am genuinely grateful for that openness, and for the chance to organise the planning of new frontend features while contributing to a more structured and professional development workflow.

5 Potential Improvements

Looking ahead, several improvements could significantly enhance the robustness, scalability, and maintainability of the AI4FT system. First, the frontend currently relies on Streamlit, which was chosen primarily for demonstration and prototyping purposes. A natural improvement would be to migrate to a more production-oriented web framework such as Django or FastAPI + React, enabling better state manage-

ment, user authentication, security controls, and long-term maintainability. A second improvement would involve leveraging a small set of annotated policy documents to fine-tune or train a local LLM specialised for European policy analysis. While the current GPT-4-family models already provide strong zero-shot performance, a domain-adapted model could improve extraction consistency, reduce hallucinations, and provide more stable classifications across Member States. Finally, a major long-term improvement would be the establishment of a full CI/CD pipeline directly integrated with the Commission's Azure VMs. Such a pipeline would allow automatic deployment of updated microservices upon merging pull requests. However, implementing this remains challenging due to the strict network and security restrictions imposed by the European Commission, which currently limit the feasibility of automated deployment workflows.